source LLM, including a Pre-Norm structure with RMSNorm (Zhang and Sennrich, 2019) function, SwiGLU (Shazeer, 2020) as the activation function for the Feed-Forward Network, Rotary Embedding (Su et al., 2024) for positional encoding, and Grouped-Query Attention (GQA) (Ainslie et al., 2023).

The Weaver family consists of models of four different sizes: Mini, Base, Pro, and Ultra, ranging from 1.8B to 34B parameters. We train different model sizes to support different applications as the complexity of writing tasks varies a lot across different domains and use cases. All Weaver models are initialized from powerful open-source LLMs. We provide detailed configurations and descriptions of Weaver models in Table 1.

2.2. Pre-training Data

We then present an overview of pre-training data selection strategies and the resulting pre-training data mixture. Since Weaver models are initialized from powerful open-source LLMs and thus already possess adequate world knowledge, the amount of continual pre-training data does not need to be super large. We consider the continual pre-training stage to be the process where Weaver learns to reallocate or re-balance its capabilities: the model allocates more capabilities to writing and content creation while reducing the capabilities on other domains such as mathematics and coding.

Therefore, we only include manually verified data sources including various kinds of content such as books, fiction, stories, news articles, papers, reports, social media copies, etc., in the pre-training data. We combine rule-based and machine-learning-based methods to filter low-quality texts. In addition to data sources and filtering, we also carefully control the data mixture between different domains. Specifically, we mix fiction data (i.e., fiction and stories) and non-fiction data (i.e., articles, papers, reports, etc.) with a ratio of 1 : 1. We also mix Chinese and English data with a portion of 4 : 1 to make Weaver supports both Chinese and English.

2.3. Training Details

We train Weaver using the standard autoregressive language modeling task where the model learns to predict the next token based on the context of previous tokens. We train Weaver models with a context length of 4096. We shuffle and merge the documents, and then truncate them to the specified context lengths to create training batches. We incorporate Megatron-Deepspeed (Shoeybi et al., 2019) and Flash Attention2 (Dao, 2023; Dao et al., 2022) to improve computational efficiency and reduce memory usage. We adopt the standard optimizer AdamW (Loshchilov and Hutter, 2017) and set the hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\varepsilon = 10^{-8}$. We use a cosine learning rate schedule with a specified peak learning rate for each model. The learning rate is decayed to a minimum learning rate of 10% of the peak learning rate. All models are trained with BFloat16 mixed precision for training stability. We present detailed pre-training configurations for each model in Table 1.

Name	Params	n_{layers}	d_{model}	n_{heads}	Context Length	Sequence Batch Size	Learning Rate	Tokens
Weaver Mini	1.8B	24	2048	16	4096	512	1e-4	50B
Weaver Base	6B	32	4096	32	4096	512	1e-4	50B
Weaver Pro	14B	40	5120	40	4096	512	1e-4	40B
Weaver Ultra	34B	60	7168	56	4096	520	5e-5	18B

Table 1 | Description for the Weaver family.

3. Data Synthesis

After pre-training, WEAVER models contain a large amount of world knowledge and writing skills and can produce human-like texts conditioning on high-quality contexts. To unlock these capabilities for real-world applications, we need to curate a high-quality dataset for alignment. The format and quality of the dataset significantly affect the coverage of abilities and the quality of aligned models. As discussed in the Introduction, the common practice for alignment data collection of existing generalist LLMs severely limits their writing capabilities. In this section, we describe our data synthesis framework in detail. We first describe the abilities we want to unlock during the alignment stage and then present our proposed data synthesis methods for both the supervised fine-tuning and the preference optimization stage.

3.1. Abilities

We first describe the categories of abilities we want to unlock for WEAVER during the alignment stage.

3.1.1. Instruction Following

The first obvious ability we need to unlock is the ability to follow writing instructions and produce human-like stylish texts. We cover various domains and tasks as listed below during data collection and alignment training.

3.1.1.1 Domains

Fiction Writing: Fiction writing refers to the abilities of models to write stories and fiction. We divide fiction writing into several subdomains with respect to the length and the genre of the fiction. We cover fiction and stories of lengths ranging from a few hundred to a few million characters, and fiction types including sci-fiction, romance, fantasy, horror, mystery, and thriller.

Creative Non-Fiction Writing: Creative non-fiction writing is a genre of writing that uses literary styles and techniques to create factually accurate narratives. We cover typical creative non-fiction writing cases including writing memoirs, biography, travelogue, journalism, social media copy, blog posts, news articles, commentary, etc.

Marketing Writing: We also consider marketing writing, which involves writing business plans, advertising copies, product promoting, marketing plans, etc. Marketing writing differs from previous categories because it is highly application-oriented and the style of generated texts is not the most important. However, marketing writing still requires human-like creativity to attract potential users.

Technical Writing: Technical writing includes tasks such as paper writing, patent writing, report writing, etc. Technical writing requires more accuracy compared to creativity. However, writing-specific training can still be helpful because it can help model produce texts that accurately adhere to the style required for specific scenarios.

3.1.1.2 Tasks

Content writing: Content writing is the basic task that requires the model to generate content (i.e., fiction, articles, etc.) based on certain instructions. Writing instructions vary in terms of whether the previous context is provided and how fine-grained the given instructions are. The task requires the LLM to be able to understand and adhere to specific requirements expressed in the instructions

while also producing texts that are consistent and coherent with previous contexts. For example, a typical content writing instruction is: "Please help me write a sci-fi about what will happen after people finally achieve AGI."

Outlining: Outlining is the task of writing outlines, which is a common practice for writers in both fiction and non-fiction writing. As discussed in the literature of long text generation (Sun et al., 2022; Yang et al., 2022a, 2023b; Zhou et al., 2019, 2023a), it is often helpful to let the model first generate an outline before generating long texts. Outlines vary according to different domains and the granularity/length of outlines. One example for the task of outlining is "Please help me write an outline of my annual work report."

Polishing & Editing: Polishing and editing require the model to improve the quality of a paragraph or rewrite it following the requirements expressed in the instructions. The task is closely related to the task of grammatical error correction (Bryant et al., 2019; Ng et al., 2014) with a key difference that the modifications are not necessarily grammatical errors. Compared to the task of academic writing polishing described in Diao et al. (2023), we support customized fine-grained control of polishing or editing requirements, which is important for human-AI interaction in AI-assisted writing systems. A typical polishing instruction may look like this: "Please help me revise the following texts, keep in mind that the revised texts should be suitable for an academic paper."

Style Transferring: The task of text style transfering requires the model to transform texts in one style into another style. For example, one may want to transform a story into a script or turn a report into a speechwriting. We cover both template-based style transfer that uses a template to provide target style information (Guu et al., 2018; Lewis et al., 2020) and description-based style transfer which uses either a keyword (Hu et al., 2017) or a short description (Zhou et al., 2023c) for the target style. For example, one may ask the model to "Transform the following book chapter into a script."

Expanding/Simplifying: Text expanding and simplifying requires the model to revise an input paragraph to make it longer or shorter according to certain instructions. Text summarization and summary-to-article generation can be regarded as two extreme cases of this task. One exemplar instruction is: "Please help me summarize this paragraph into one sentence.".

Brainstorming: Brainstorming requires the model to help users come up with creative ideas based on the current context and user instructions. A typical brainstorming instruction is: "Please give me 5 possible character descriptions for a villain to appear in the next chapter, including his name, appearance, occupation, and background."

Reviewing: Reviewing refers to the task of reading and analyzing a given piece of text critically and then producing comments or revising suggestions. For example, one may ask the model to "Please take a look at my essay and list 5 suggestions to improve it."

3.1.2. Instruction Annotation

We also train WEAVER to support the instruction annotation task. As described in Humpback (Li et al., 2023) and LongForm (Köksal et al., 2023), given a piece of text, the task requires the model to generate an instruction to which the input texts may be the answer. However, vanilla instruction backtranslation only supports the writing task. Therefore, for instruction annotation, we require the model to synthesize an instruction-response pair based on a text span. The response can be the text span, a part of the text span, or inferred from the text span. This substantially broadens the scope for vanilla instruction backtranslation since most automatically mined text spans may not be suitable for a certain instruction on itself while a part of the text span can be a valid response or one may construct a high-quality instruction-response pair based on it. The instruction annotation